XM · CLASE 7

Soluciones de Analítica con Databricks

Clase 7: Oro: features para pronosticar la demanda

Docente: Manuela Larrea Gómez

Agenda de la clase 7

La teoría vive en el notebook: se recorre, se ejecuta y se discute sobre tus datos

01 — 10 MIN

Encuadre y sector

Qué construimos (features_demanda_diaria) · qué mueve la demanda en Colombia: festivos, hidrología, temperatura · hito 4

02 — 50 MIN

Notebook guiado, con la docente

Oro y contrato · calendario y festivos · lags y ventanas · fuga de información y horizonte · banderas · MERGE y llave primaria

03 — 50 MIN

Lab 7: el mismo notebook, solo

agregar_calendario y agregar_lags en src · las dos preguntas · verificar() · commit y PR

04 — 10 MIN

Cierre

Hito 4 abierto · tag s07 · así sería en Azure · próxima clase: código, configuración y tests

El notebook de hoy: siete secciones con teoría, gráfico y código

clases/c07-features-gold/notebooks/lab07_features_gold · widget usuario · requiere demanda_diaria (lab 6)

#	Sección	Teoría que se explica	Lo que se ejecuta / se ve
1	Oro y contrato	Qué es oro, qué es una feature, la regla de t	72.704 serie-días, 355 series; contrato de la tabla
2	Calendario y festivos	Fin de semana −10 %, festivo −12 %; Ley 51/1983; dato de referencia versionado	Promedio por día de semana en SQL; festivos_co.csv (55 filas)
3	TODO 1	dayofweek 1=dom…7=sáb; left join; nulo si el año no está cubierto	agregar_calendario en build.py; celda de prueba
4	Lags y ventanas	Window: partitionBy serie, orderBy fecha, rowsBetween(-n, -1)	TODO 2 agregar_lags; prueba de mesa de media_7
5	Fuga y horizonte	Tres fugas: ventana con t, ventana centrada, versión futura; y_h7 = lead(7)	media_7 correcta vs. con fuga; pregunta
6	Banderas	Plata no inventa; oro describe: es_cero, dias_con_datos	Series cortas y con ceros
7	Construir y publicar	.transform encadenado; MERGE por llave; feature table (PK); Federation	Dos MERGE; historial; PRIMARY KEY, CLUSTER BY, comentarios; pregunta

Dos TODO en src/xm_demanda/features/build.py: agregar_calendario() y agregar_lags(). Lags, ventanas y horizonte vienen de conf/base.yml.

Sector: qué mueve la demanda eléctrica en Colombia

Cuatro variables que un pronóstico a 7 días debe conocer, y cuál entra hoy

−12 %
un festivo entre semana frente a un día laboral normal (en nuestros datos 2026)

212 vs 238
GWh promedio un domingo vs. un lunes en el SIN, 2026

19
festivos en 2026 (uno nuevo: Chiquinquirá, 13 de julio); muchos van al lunes por la Ley 51 de 1983

≈ 70 %
de la generación del SIN es hidráulica: El Niño mueve oferta y precio, no tanto la demanda

Calendario

Día de la semana, festivos, puentes. Conocido de antemano, sin costo, y explica la mayor parte de la variación semanal. Entra hoy.

Temperatura

Relevante en la costa Caribe (aire acondicionado), casi nula en Bogotá. Se prueba en la clase 9 con evidencia, no por intuición.

Autogeneración y largo plazo

La solar distribuida resta demanda visible al mediodía. La UPME proyecta a años; nosotros a 7 días: distinto problema, distintas variables.

Fuente: Ley 51 de 1983 (funcionpublica.gov.co) · XM, demanda real del SIN 2026 (nuestra tabla demanda_diaria) · UPME, proyecciones de demanda · XM, informes de operación (hidrología)

Tres ideas para llevar al notebook

Todo lo demás es código; esto es criterio

Fuga de información: una feature solo puede usar lo que existía en t

Dos fugas típicas en series de tiempo: mirar el futuro por la ventana, y mirar una versión del dato que todavía no se había publicado

Fuga 1 · la ventana incluye hoy o el futuro

"media_7" calculada con rowsBetween(-3, 3) o incluyendo la fila actual

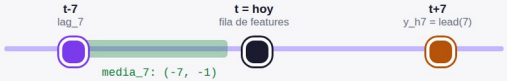



```
Window.partitionBy(*LLAVE_SERIE).orderBy("fecha")
      .rowsBetween(-3, 3)        # usa t+1..t+3: fuga
      .rowsBetween(-6, 0)        # incluye hoy: y_h1 se filtra
```

El modelo aprende del futuro: métrica excelente en entrenamiento, inservible en producción

Correcto · la ventana termina en t-1

lag_k = valor en t-k · media_n = promedio de t-n … t-1 · objetivo = valor en t+7




```
w = Window.partitionBy(*LLAVE_SERIE).orderBy("fecha")
lag_7   = F.lag("demanda_real_kwh", 7).over(w)
media_7 = F.avg("demanda_real_kwh").over(w.rowsBetween(-7, -1))
y_h7    = F.lead("demanda_real_kwh", 7).over(w)   # solo para entrenar
```

Lo que el modelo ve en t es exactamente lo que verá en producción en t

Fuga 2 · una versión que aún no existía

El ajuste del 5 de agosto corrige el 20 de julio. Si entreno "como si" en julio ya lo supiera, el modelo ve datos que XM no había publicado




```
# plata hoy: siempre la última versión
demanda_diaria → 1.259,2 (vigente)
# point-in-time: la versión vigente EN esa fecha
bronce + VERSION AS OF / _publication_date <= t
# hoy lo anotamos como decisión (ADR); clase 9 lo resuelve
```

Grave cuando las correcciones son grandes; en nuestros datos las republicaciones cambian ≈ 1–2 %

Regla para cada feature: **¿este número existía y estaba publicado el día t?** Si la respuesta es no, es fuga. Las ventanas terminan en t-1, el objetivo es lead(h), y la versión del dato se documenta.

1 · Una feature vive en t

Solo información que existía el día t. El objetivo y_h7 es la única excepción.

2 · Por serie, por fecha, hasta t−1

Window.partitionBy(serie).orderBy(fecha).rowsBetween(−n, −1)

3 · La fuga no da error

Entrena espectacular, falla en producción. Se previene con diseño, no con try/except.

Free Edition vs. Azure Databricks: qué simulamos hoy

Las funciones y el contrato son los mismos; cambia quién las ejecuta y de dónde vienen las fuentes

Elemento	Hoy en Free Edition	En XM sobre Azure
Construcción de features	Tú, desde el notebook, en tu esquema	Tarea gold_features del job diario (clase 12), después del pipeline de plata
Feature table	Tabla Delta con PRIMARY KEY en tu esquema	Misma tabla en gold_energia; el entrenamiento la registra (clase 9) y el serving busca por llave (clase 11)
Festivos	CSV versionado en data/reference	Igual (cambia poco y se revisa en PR); o tabla de referencia gobernada
Temperatura, hidrología	No entran hoy (se prueban en la clase 9)	Lakehouse Federation: se consultan en el SQL Server / Postgres del área sin copiarlas
Point-in-time	Extensión opcional del lab	Feature Engineering en UC con timestamp key; entrenamiento con lookups point-in-time

Lab 7: el mismo notebook, solo, con dos TODO en src

Después del recorrido con la docente, cada uno lo ejecuta en su esquema de principio a fin

1. **Secciones 1–2**
   Ejecutar y leer. Confirmar el patrón semanal sobre tus datos
2. **Sección 3: agregar_calendario()**
   En build.py; la celda de prueba la valida (≥ 8 festivos marcados)
3. **Sección 4: agregar_lags()**
   En build.py; prueba de mesa: media_7 = promedio manual
4. **Secciones 5–7**
   Las dos preguntas por escrito; dos MERGE; PK, CLUSTER BY, comentarios
5. **verificar() · commit · PR**
   feat(c07): features de oro con calendario, lags y ventanas

Criterios de aceptación (hito 4 abierto)

- 72.704 filas, llave única, calendario sin nulos
- lag_7 nulo solo en los primeros 7 días; y_h7 solo en los últimos 7
- media_7 nunca incluye t (verificar() lo recalcula)
- Dos MERGE; PRIMARY KEY; CLUSTER BY; comentarios
- test_features.py en verde; verificar() en OK; PR en verde

Comparar con s07

github.com/<docente>/Databricks-to-XM/compare/s07...<tú>:Databricks-to-XM:main · o git diff s07 -- src/xm_demanda/features clases/c07-features-gold/notebooks

Duración: 50 minutos

Así sería en Azure Databricks: oro como feature table de producción

Hoy	En XM
construir_features() lanzada desde el notebook	notebooks/03_gold_features.py como tarea del job diario, con catalog=prod; alerta si es_festivo queda nulo (año sin festivos)
Lags y ventanas en conf/base.yml	Igual: cambiar el experimento es un PR a conf, no a src; el bundle (clase 13) lo despliega por ambiente
PRIMARY KEY declarada a mano	Feature table de Unity Catalog: linaje de qué features usó cada modelo; lookups por llave en serving
Festivos desde un CSV del repo	Igual, o tabla de referencia gobernada por grp_analitica con MERGE anual
Temperatura: no entra	Lakehouse Federation al servidor del área de operación; una función agregar_temperatura() con su test

Lo que no cambia: la regla de t, las ventanas hasta t−1, la llave, el MERGE idempotente y las funciones en src con sus tests.

Resumen de la clase 7 y preparación para la clase 8

Logros de hoy

- **Oro responde una pregunta**
  Demanda a 7 días: cada columna tiene un porqué
- **El calendario explica mucho**
  Fin de semana −10 %, festivo −12 %; festivos versionados en el repo
- **Lags y ventanas por serie**
  Window hasta t−1; nunca incluyen t
- **La fuga no da error**
  Tres formas; se previene con diseño y con verificar()
- **Hito 4 abierto**
  features_demanda_diaria con MERGE, PK y contrato

Próxima clase

- **Código, configuración y datos separados**
  Notebooks delgados, paquete xm_demanda, conf por ambiente
- **Tests e integración continua**
  pytest, ruff, pre-commit; un PR que rompe un test
- **Reproducibilidad ante auditoría**
  Reconstruir hoy el pronóstico de hace seis meses
- **Traer a la clase 8**
  Lab 7 terminado y PR en verde: la clase 8 trabaja sobre tu fork



iData
Global | Data & AI

Gracias

Nos vemos en la clase 8: Código, configuración y datos separados

DOCENTE

Manuela Larrea Gómez